

● Pro Coffee Gear – Agent System Guide

Four separate views of the system: how knowledge moves, how people are onboarded, how skills/scripts/tools are shared, and how the fleet is routed and monitored.

EXECUTIVE SUMMARY · WHY THIS MATTERS

Give every manager the power to build and continually improve the systems their function runs on

PCG already produces valuable knowledge every day: why a return happened, which vendor issue keeps recurring, what customers object to, where margin leaks, and which manual process slows a team down. Today that knowledge is spent twice: once living the problem, and again translating it – into instructions for someone running a manual workflow, or a spec for a developer who wasn't there. This architecture removes the translation step: the manager who understands the problem works directly with an agent that remembers the function's history, builds the fix as a real system, and keeps improving it – with ownership, permissions, source code, and health visible.

● Institutional knowledge becomes usable

Honcho gives the function a shared memory of PCG's rules, systems, exceptions, prior decisions, and recurring failure patterns. A new project starts with accumulated context instead of a blank chat. A new teammate can inherit the team's operating knowledge while each contributor remains identifiable and raw chats are not copied into a shared feed.

● Managers solve their own problems – no translation loss

Hermes gives each teammate a private builder. The person who owns the problem diagnoses it, drafts the workflow, writes the script, builds the dashboard, and tests it in one conversation – no handoff, no requirements document, no waiting on someone else's queue. GitHub preserves the source and rollback path. Cloudflare can publish approved tools at a live URL. Notion records the owner, audience, status, and health.

● Systems keep improving instead of decaying

A system built this way is never finished – it is versioned. When reality changes (a new vendor quirk, a failure pattern, a better rule), the owner tells the agent and the tool is updated the same day. Useful skills and scripts publish forward to the next eligible teammate; material lessons reach LT through summaries. The next person starts from the last person's progress.

One motion, every function: a recurring problem becomes a system that runs without you

Customer service: a complaint that arrives three times stops being three tickets. It becomes a Front workflow that assembles the case, drafts the reply, and flags the product or vendor behind it.

Merch and Growth: pricing drift and margin leaks stop hiding in spreadsheets. Monitors watch them continuously, and real customer questions become the content plan.

Operations: already proven – the open-box process went from spreadsheet wrangling to a live dashboard built exactly this way. Vendor-claim aging and inventory exceptions are the same motion, waiting their turn.

Finance: an authorized-but-unpaid order sits invisible today until someone happens to look. A built check finds it daily – and each exception teaches the next check.

Sales: a stalled quote is a signal, not a loss. Follow-up fires on time, leads rank themselves, and every lost-deal reason accumulates into sharper positioning.

Leadership: LT stops asking "what's happening?" Material changes, decisions needed, and cross-function risk arrive as a daily brief – without reading anyone's chats.

The compounding loop

Work creates conversations. Conversations create memory. Memory improves the next diagnosis and build. Approved builds become versioned systems whose results create new evidence; when the team reviews that evidence with the agent, the lesson enters memory and the system gets upgraded. The tenth tool a function builds ships faster and fits better than the first, because it starts from everything the function has already learned.

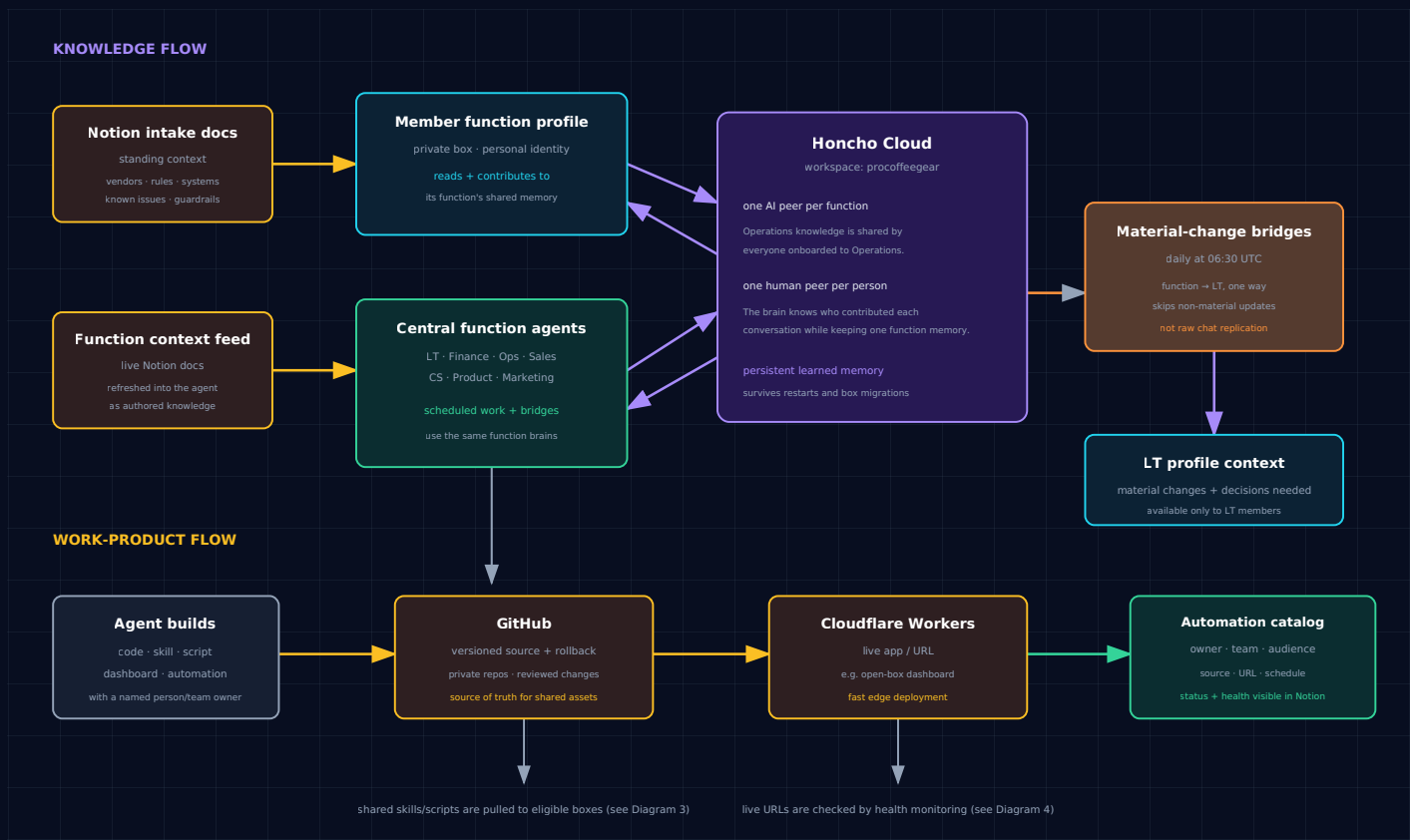
Builder capacity with clear ownership

Agents do not replace function owners or approve critical changes. They give owners better memory and more building capacity. Permissions control who receives each profile; Git records changes; Notion makes ownership visible; monitoring verifies whether shared systems are actually working.

The durable advantage: managers who build and continually improve their own systems on compounding knowledge – gains PCG keeps when the next problem, project, or teammate arrives.

Diagram 1 – Data flow after onboarding

This diagram intentionally excludes onboarding. It shows the steady-state movement of knowledge and built work.



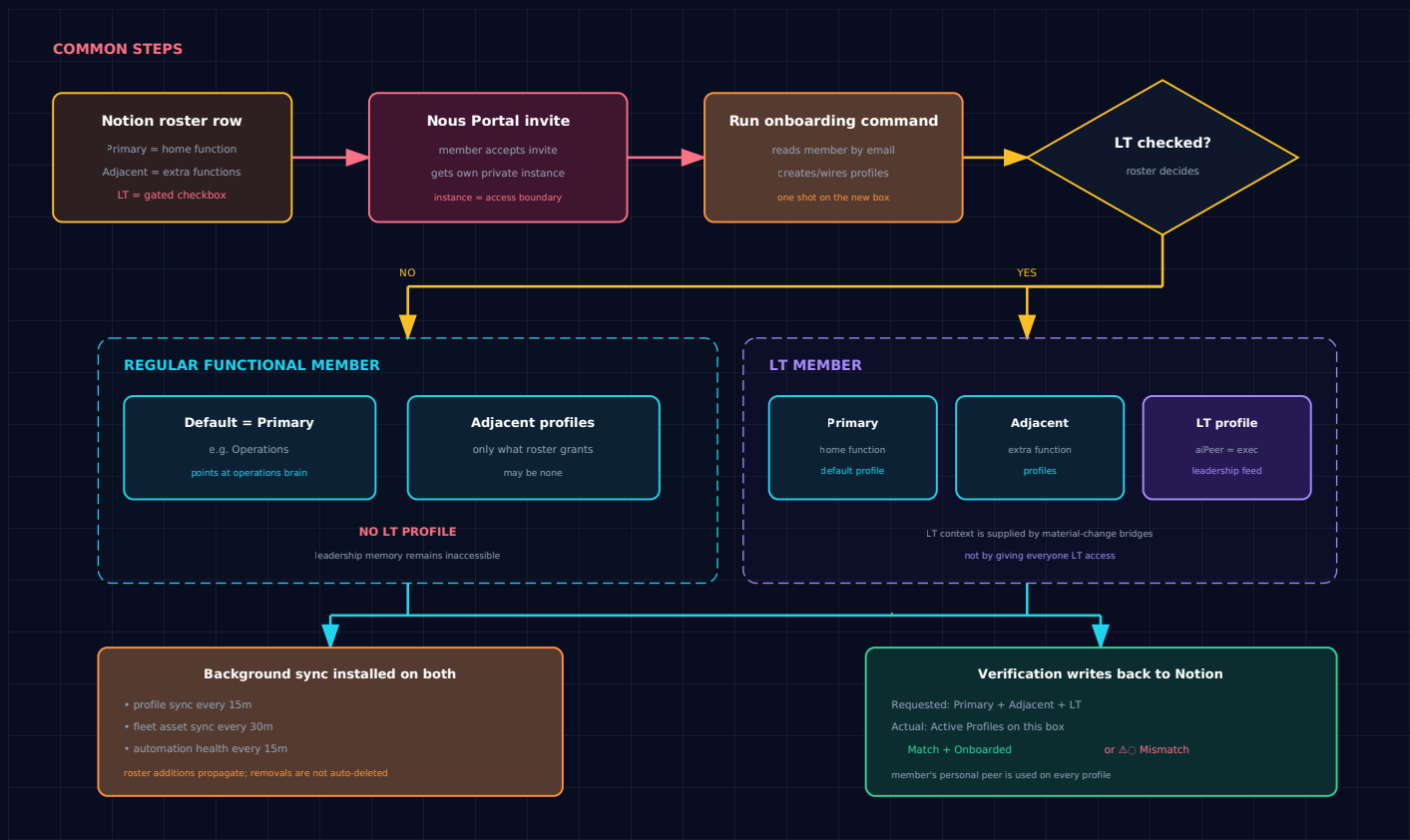
Knowledge is not one giant pool. Each function has its own shared brain. Bridges move only the relevant material signal between functions and into LT.

People remain identifiable. Everyone in a function uses the same AI peer, but Honcho remembers each contributor as their own human peer.

Built work has a separate lifecycle. Memory goes to Honcho; source goes to GitHub; live apps go to Cloudflare; significant systems are indexed in Notion.

Diagram 2 – Onboarding: functional member vs. LT member

Both people get their own private instance. The LT checkbox changes which profiles and leadership context are installed.



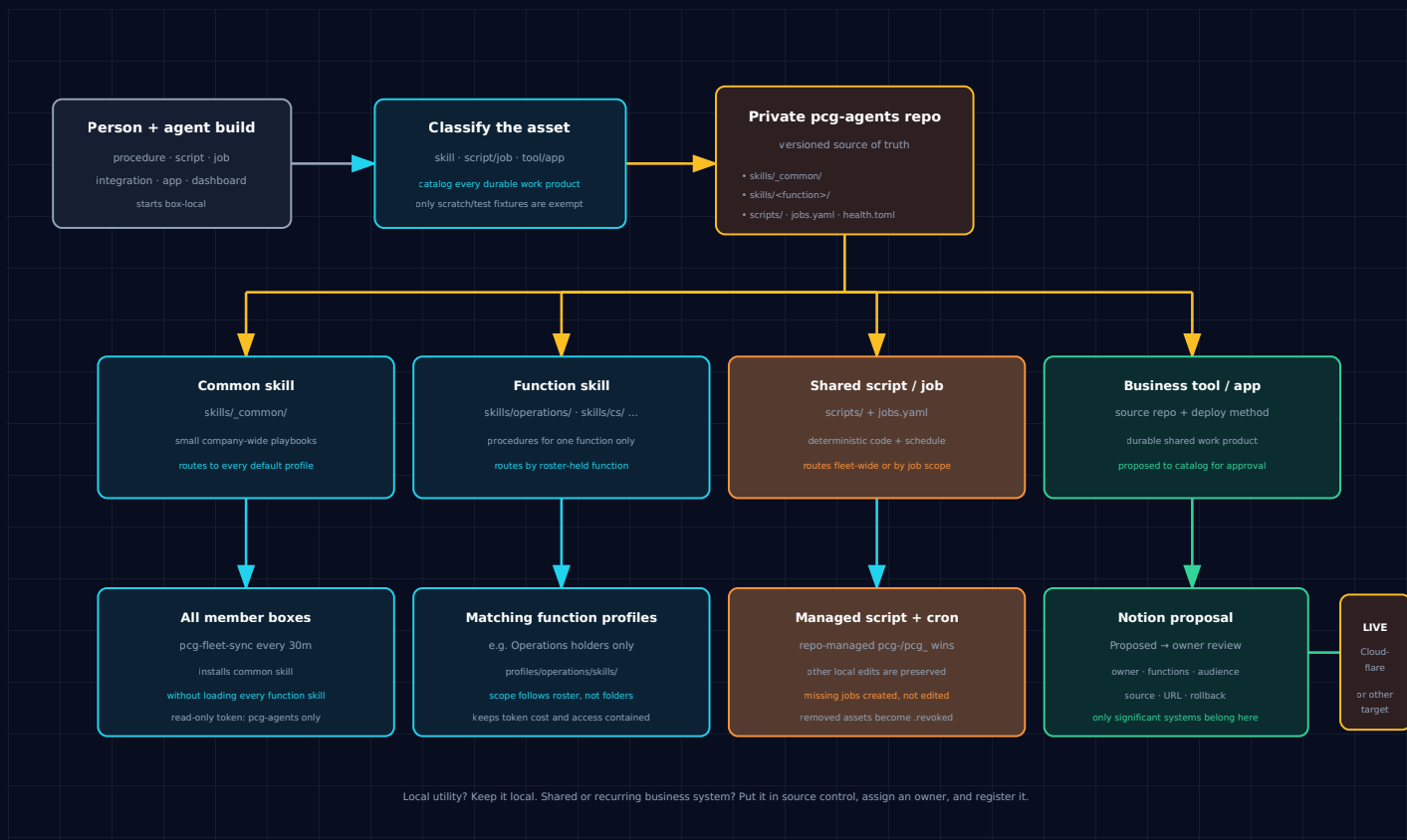
Primary is the home function. It becomes the box's default profile; it is no longer hardcoded to LT.

Adjacent means extra functional access. Newly added Adjacent profiles appear through the 15-minute profile sync.

LT is an explicit gate. Only checked roster rows receive the LT profile and leadership memory.

● Diagram 3 – Skill, script, job, and tool sharing

Box-local work is not shared automatically. The private pcg-agents repo is the distribution channel; Notion is the catalog and curation layer.



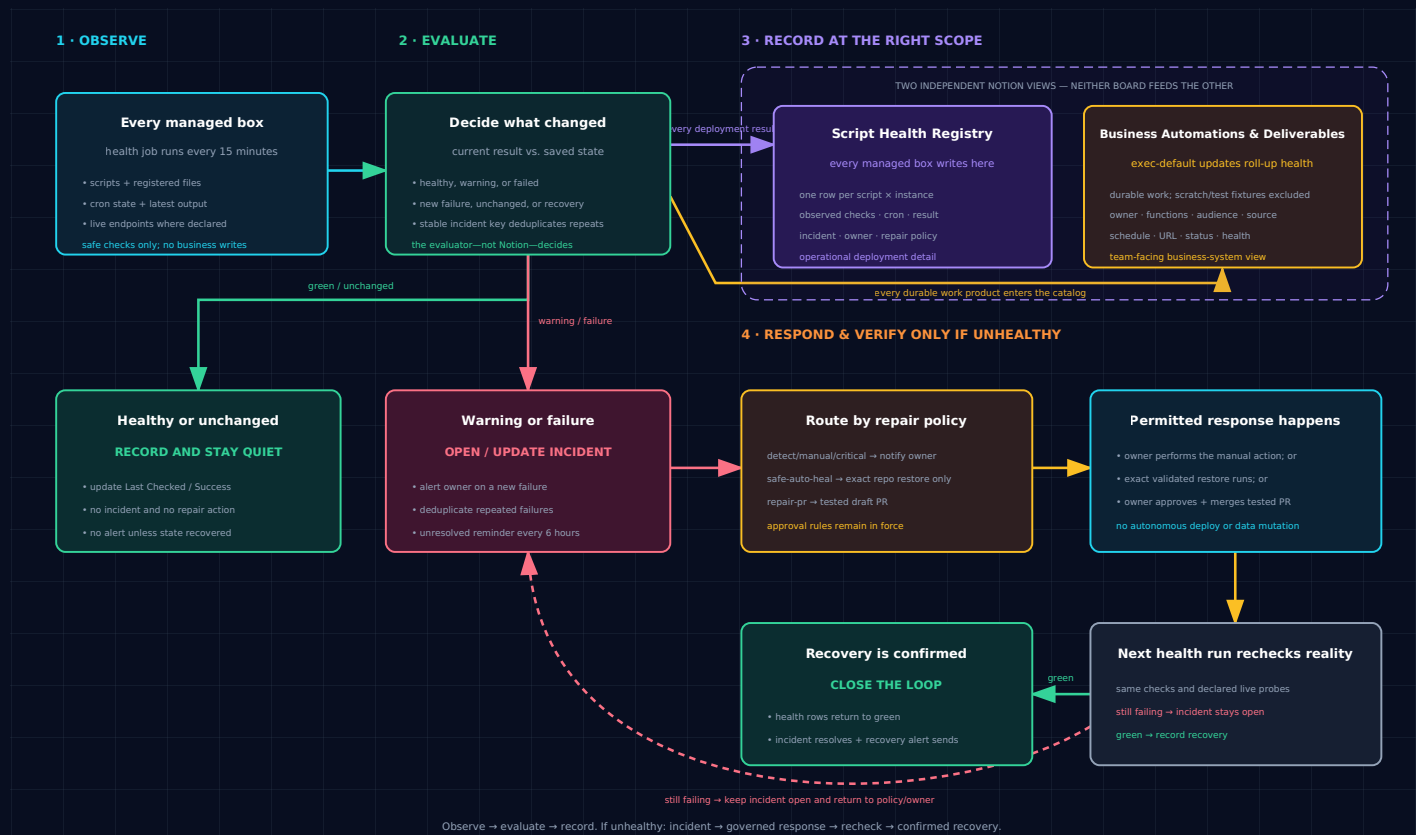
Skills teach the agent how to work. They are loaded only into the profiles that need them, so Operations procedures do not clutter CS.

Scripts do deterministic work. Shared scripts self-update from GitHub; managed jobs can be declared in jobs.yaml and run without model cost.

Tools are business deliverables. Dashboards, apps, integrations, and recurring automations get an owner, team relationship, source, status, health, and rollback method.

● Diagram 4 – Observe, evaluate, record, respond, and verify

Each managed box checks its own deployments. The result is evaluated first, then recorded at the right scope. Only warnings or failures enter a governed response path; a later green check closes the incident.



One monitor, two board scopes. Every box writes instance-specific script health. Only exec-default updates the global health of significant business deliverables.

The boards do not feed each other. Together they answer what exists, who owns it, which people/functions use it, where it runs, and whether each deployment is working.

Repair remains governed. Detection is automatic. Safe restoration is deliberately narrow; PRs require owner approval; critical and manual systems never change themselves.

● How the four views fit together

- ▶ **Onboarding establishes access and routing.** The roster decides the member's home function, adjacent functions, and whether LT is permitted. The member's box is then wired to the corresponding shared brains.
- ▶ **Data flow begins after onboarding.** Function work teaches the function brain. Bridges summarize material changes into the LT feed. Built source and deployed tools follow their own GitHub/Cloudflare lifecycle.
- ▶ **Asset sharing turns one person's build into team capability.** Skills route to the profiles that need them, scripts route to managed boxes, and every durable work product is cataloged and assigned to an owner.
- ▶ **Monitoring closes the loop.** The two Notion boards show what exists and whether it is healthy; recurring checks detect drift, notify owners, and verify recovery.

● What must be cataloged

Every durable app, dashboard, automation, integration, report, document, model, spreadsheet, shared skill, and scheduled job enters Business Automations & Deliverables as **Proposed**. Temporary scratch files and throwaway test fixtures are the only exemptions. A function owner then approves, rejects, or requests changes.

● Current operating truth

- ▶ **Shared by nature:** Honcho memory, Notion pages/boards, GitHub repositories, and Cloudflare URLs.
- ▶ **Box-local until distributed:** skills, scripts, cron jobs, profiles, tokens, and standing-context files.
- ▶ **Additions are automated; destructive removal is guarded.** New Adjacent profiles are created automatically. Removed repo assets are quarantined as .revoked. Removing a roster permission does not silently delete a local profile.
- ▶ **"Healthy" is evidence-based.** The monitor checks syntax, declared dependencies, cron state/output, live endpoints, and missing previously registered scripts. It does not execute dangerous external writes merely to prove health.

● Appendix A – Definitions

- **Hermes** – the agent platform team members work in.
- **Instance / box** – one member’s isolated Hermes server and access boundary.
- **Profile** – a function-specific agent identity on a box, with its own context, tools, and skill index.
- **Primary** – the roster-selected home function; it becomes the member box’s default profile.
- **Adjacent** – extra functional profiles a member is allowed to switch into.
- **LT checkbox** – the explicit roster gate for receiving the LT profile and leadership context.
- **Honcho** – the persistent shared-memory layer.
- **AI peer** – one function’s shared brain in Honcho.
- **Human peer** – one person’s identity in Honcho, used across their permitted profiles.
- **Standing context** – authored function knowledge loaded from Notion.
- **Learned memory** – durable knowledge learned from real conversations.
- **Bridge** – a one-way summarization route between function brains or into LT.
- **Skill** – a procedural playbook that teaches a profile how to perform a recurring type of work.
- **Script** – deterministic code used by an agent or scheduled job.
- **Tool / app** – a business-facing deliverable such as a dashboard, integration, or deployed workflow.
- **pcg-fleet-sync** – the zero-token 30-minute puller that distributes scoped skills, scripts, and managed job definitions from GitHub.
- **Repo-managed asset** – a `pcg-/pcg_-`prefixed item where the repository wins conflicts.
- **Quarantine / .revoked** – a safe rename used when a previously distributed asset disappears from the repo; it is not hard-deleted.
- **Business Automations & Deliverables** – the team-facing Notion catalog of significant systems and builds.
- **Script Health Registry** – the operational Notion board with one row per script deployment/instance.
- **Repair policy** – the declared rule for a failure: detect-only, safe-auto-heal, repair PR, critical approval, or manual.
- **Incident key** – a stable fingerprint that deduplicates repeated reports of the same failure.
- **Cloudflare Workers** – the live deployment layer for web tools such as the open-box returns dashboard.
- **Vendor lock-in** – dependency on one supplier that makes leaving expensive; this architecture keeps code, memory, control data, and deploys separable.

● Appendix B — FAQ

Does every employee get an LT profile?

No. LT is explicitly gated by the roster checkbox. Regular functional members receive Primary plus any Adjacent profiles only.

Does everyone in Operations share Sina's personal memory?

No. Everyone assigned to Operations shares the Operations AI peer—the function brain—but each person is represented by their own human peer.

Are all chats copied into LT?

No. Daily bridges ask each function brain for material changes and decisions leadership should know. Non-material updates and raw chats are not copied wholesale.

How does a skill reach the team?

It is committed to either `skills/_common/` or a function folder in the private repo. The 30-minute fleet sync places it only into eligible profiles.

How does a script reach the team?

Shared scripts live under `scripts/` in the private repo and are pulled to managed boxes. A missing `pcg_` job declared in `jobs.yaml` is created automatically.

Does the repository overwrite a person's local work?

Only repo-managed `pcg_`/`pcg_` assets are authoritative. Other local changes are preserved and reported as conflicts.

Where can I see everything the team has built?

The Business Automations & Deliverables board shows every durable work product, its owner, functions, audience, source, schedule, URL, status, and health. Only scratch files and throwaway test fixtures are excluded.

Where can I see whether the underlying code is working?

The Script Health Registry shows each script deployment/instance, checks performed, cron relationships, latest success or failure, incident state, and repair policy.

What happens when something breaks?

The 15-minute health job updates the boards and alerts on failure or recovery. Unresolved failures remind every six hours. The repair policy determines whether the owner handles it manually or a tested repair PR may be opened for approval.

Can the repair system change critical infrastructure by itself?

No. Critical-approval and manual policies prevent autonomous changes. Even repair-PR items require the owner to approve and merge the fix.

How do we undo a bad change?

The source and rollback method are recorded. Most shared assets roll back by reverting the Git commit; fleet sync or redeployment restores the previous version.

What if onboarding shows Mismatch?

Compare Requested versus Active Profiles in the roster. Do not repeatedly rerun onboarding; send the mismatch detail to the system owner so the failed step can be diagnosed.